

**Implementasi Perbandingan Model Klasifikasi KNN, Naïve Bayes
Decision Tree, SVM untuk Deteksi Dini Penyakit Jantung dengan
Pendekatan Machine Learning.**

Diajukan Untuk Memenuhi UTS Mata Kuliah Pembelajaran Mesin

Dosen Pengampu : Junta Zeniarja M.Kom



Disusun oleh :

Nama : Muhammad Syafiq Al Fajri

NIM : A11.2024.15698

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS DIAN NUSWANTORO**

DAFTAR ISI

DAFTAR ISI	2
BAB 1 PENDAHULUAN	3
1.1 Latar Belakang Masalah.....	3
1.2 Tujuan Projek.....	3
1.3 Relevansi OBE (Outcome-Based Education)	3
BAB 2 STUDI LITERATUR.....	4
2.1 Tinjauan Paper Utama	4
2.2 Sintesis Metode Optimasi.....	4
BAB 3 AUDIT DATA.....	5
3.1 Sumber Data dan Karakteristik	5
3.2 Data Dictionary.....	5
3.3 Etika, Privasi, dan Batasan Penggunaan.....	5
3.4 Audit Data dan Preprocessing	6
3.5 Strategi Pembagian Data (Data Splitting).....	8
3.6 Model Baseline dan Algoritma yang Diuji.....	9
3.7 Hyperparameter (Grid Search)	9
3.8 Cross Validation dan Matrik.....	9
BAB 4 HASIL EKSPERIMEN.....	11
4.1 Tabel Perbandingan Semua Model	11
4.2 Confusion Matrix Model Terbaik (KNN Tuned)	11
4.3 Learning curve untuk KNN_tunning	11
4.4 Error Analysis	12
BAB 5 PEMBAHASAN.....	13
5.1 Pemilihan Model Terbaik.....	13
5.2 Trade-off Akurasi-Kompleksitas-Interpretabilitas	13
5.3 Implikasi Institusi.....	13
BAB 6 DESAIN APLIKASI	14
6.1 Arsitektur Backend	14
6.2 Alur UI/UX Streamlit.....	14
6.3 Cara Menjalankan.....	14
BAB 7 KESIMPULAN DAN SARAN.....	15
7.1 Capaian Outcome.....	15
7.2 Keterbatasan	15
7.3 Rekomendasi Pengembangan.....	15
Lampiran.....	16
DAFTAR PUSTAKA	20

BAB 1

PENDAHULUAN

1.1 Latar Belakang Masalah

Penyakit jantung, atau yang dikenal sebagai penyakit kardiovaskular, merupakan kondisi malfungsi pada bagian-bagian jantung seperti katup, lapisan, pembuluh darah, dan otot—yang umumnya disebabkan oleh penyumbatan arteri, peradangan, infeksi, maupun cacat lahir. Kasus penyakit ini terus meningkat setiap tahunnya; setidaknya 15 dari 1.000 orang, atau sekitar 2.784.064 jiwa di Indonesia, menderita penyakit ini. Berdasarkan survei *Sample Registration System* (SRS) tahun 2014, penyakit jantung koroner menjadi penyebab 12,9% kematian di Indonesia, dengan tingkat kejadian mencapai 1,5% secara nasional dan 1,6% di wilayah Jawa Barat (Syahida Nafisah, Novianti Nuril Inayah, Baharuddin Yusuf, 2024). Tingginya angka kematian tersebut dipicu oleh dua faktor utama, yaitu faktor internal (seperti genetik, gaya hidup buruk, pola makan tidak sehat, tingkat stres, hipertensi, hingga gangguan mental) serta faktor eksternal (seperti lingkungan yang tidak sehat dan kurangnya dukungan dari keluarga).

1.2 Tujuan Projek

Proyek ini bertujuan membangun sistem deteksi dini penyakit jantung berbasis Machine Learning yang mampu memprediksi probabilitas risiko secara cepat dan objektif. Secara khusus, proyek ini membandingkan kinerja model K-Nearest Neighbors (KNN), Naive Bayes, dan Decision Tree dalam mengklasifikasikan pasien berisiko penyakit jantung setelah menerapkan penanganan missing values dan normalisasi fitur. Evaluasi performa dilakukan menggunakan Confusion Matrix serta metrik akurasi, presisi, recall, dan F1-score untuk menentukan model yang paling sesuai bagi kebutuhan klinis.

1.3 Relevansi OBE (Outcome-Based Education)

Penelitian ini sejalan dengan prinsip *Outcome-Based Education* (OBE) karena menghasilkan capaian keterampilan teknis dan non-teknis yang terukur melalui penyelesaian masalah di dunia nyata. Melalui proyek ini, keterampilan praktis ditingkatkan dengan merancang sistem deteksi dini penyakit jantung, yang mencakup proses penanganan data hilang dan normalisasi. Penerapan algoritma *machine learning* seperti KNN, Naive Bayes, SVM, dan Decision Tree juga dipelajari sekaligus dievaluasi keandalannya menggunakan *Confusion Matrix* beserta metrik akurasi, presisi, *recall*, *F1-score*, dan *balanced accuracy*. Penelitian ini turut menanamkan nilai etika profesional seperti menjaga keabsahan data dan privasi pasien yang menjadi capaian penting dari tujuan program studi.

BAB 2

STUDI LITERATUR

2.1 Tinjauan Paper Utama

Penulis	Tahun	Judul	Jurnal	DOI	Relevansi
Amine et al.	2024	Early heart disease prediction using feature engineering and machine learning algorithms	Heliyo	10.1016/j.heliyon.2024.e38731	Dasar penggunaan KNN dan SVM dengan optimasi fitur.

2.2 Sintesis Metode Optimasi

Berdasarkan literatur di atas, proyek ini mengintegrasikan lima metode optimasi strategis:

1. **Hyperparameter Tuning:**

Menggunakan GridSearchCV untuk mengoptimasi parameter sensitif seperti nilai pada KNN dan kernel pada SVM guna mencegah overfitting.

2. **Imbalance Handling (SMOTE):**

Mengatasi bias prediksi pada kelas mayoritas agar model lebih sensitif dalam mendeteksi kelas 'Sakit'.

3. **Stratified Cross-Validation:**

Menjamin stabilitas evaluasi model pada dataset medis yang terbatas melalui pembagian lipatan data yang proporsional.

4. **Feature Engineering & Scaling:**

Penggunaan StandardScaler sangat krusial karena model utama (KNN & SVM) bekerja berdasarkan perhitungan jarak Euclidean.

5. **Model Analysis (Analisis Error):**

Mengidentifikasi pola *False Negative* secara mendalam untuk memahami batasan teknis model dalam skenario klinis nyata.

BAB 3

AUDIT DATA

3.1 Sumber Data dan Karakteristik

Deskripsi Dataset :

1. Sumber : <https://data.mendeley.com/datasets/jtwbww4z9k/1>
2. Contributor : Sherko MURAD
3. DOI : 10.17632/jtwbww4z9k.1
4. Licence : CC BY 4.0
5. Mendeley : MURAD, Sherko (2025), "Heart Disease", Mendeley Data, V1, doi: 10.17632/jtwbww4z9k.1
6. Sampel : 1.190 baris, 629 data (52,9%) memiliki label 1 dan 561 data (47,1%) memiliki label 0, 12 kolom (11 fitur + 1 target).
7. Target : 0 = Sehat, 1 = Penyakit Jantung Terdeteksi

3.2 Data Dictionary

Data kesehatan yang disajikan dalam tabel tersebut mencakup berbagai variabel medis yang dikelompokkan berdasarkan jenis datanya. Untuk hasil laboratorium yang bersifat angka pasti, terdapat fitur:

Fitur	Tipe Data
age	Numerik kontinu
sex	Kategoris biner
chest pain type	Kategoris ordinal
resting bp s	Numerik kontinu
cholesterol	Numerik kontinu
fasting blood sugar	Kategoris biner
resting ecg	Kategoris nominal
max heart rate	Numerik kontinu
exercise angine	Kategoris biner
oldpeak	Numerik kontinu
ST slop	Kategoris ordinal

3.3 Etika, Privasi, dan Batasan Penggunaan

- a. Etika & Privasi, Dataset ini bersifat anonim (*de-identified*). Tidak terdapat informasi identitas pribadi (PII) seperti nama, alamat, atau nomor identitas pasien, sehingga penggunaan data ini memenuhi standar privasi riset medis.
- b. Lisensi, Data digunakan di bawah lisensi **CC BY 4.0**, yang memungkinkan penggunaan untuk tujuan akademik dengan memberikan atribusi kepada kontributor asli.

- c. Batasan penggunaan, Model ini didesain sebagai alat bantu pendukung keputusan (*Clinical Decision Support System*) dan bukan pengganti diagnosis medis profesional. Keputusan akhir harus tetap divalidasi oleh dokter spesialis jantung.

3.4 Audit Data dan Preprocessing

a. Audit Data

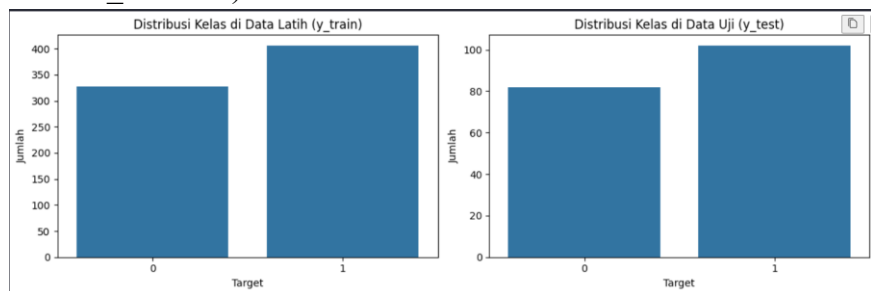
Sebelum dilakukan pemodelan, audit dilakukan untuk memastikan kualitas data:

- **Kesalahan Tipe Data:** Kolom oldpeak awalnya terdeteksi sebagai objek (string) karena penggunaan koma sebagai desimal. Telah dikonversi menjadi float agar dapat diolah secara matematis.
- **Nilai Tidak Wajar:** Ditemukan nilai 0 pada kolesterol (172 data) dan resting bp s (1 data) yang secara klinis mustahil bagi pasien hidup.
- **Duplikasi:** Ditemukan 272 baris data duplikat yang berisiko membuat model mengalami *overfitting*.
- **Missing Values:** Tidak ditemukan nilai kosong (null) eksplisit, namun nilai 0 pada fitur tertentu diidentifikasi sebagai *implicit missing values*.

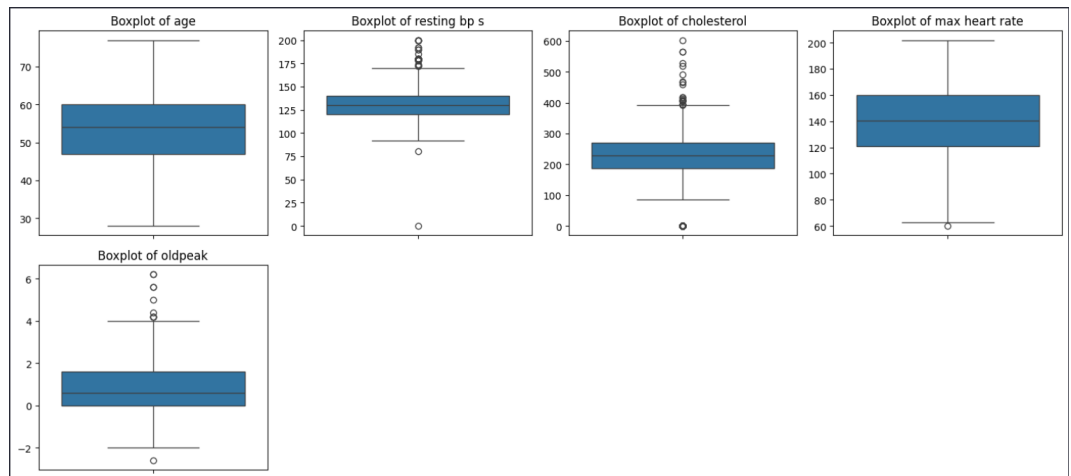
b. Pipeline Preprocessing

Berikut adalah langkah teknis untuk setiap tahap pembersihan data:

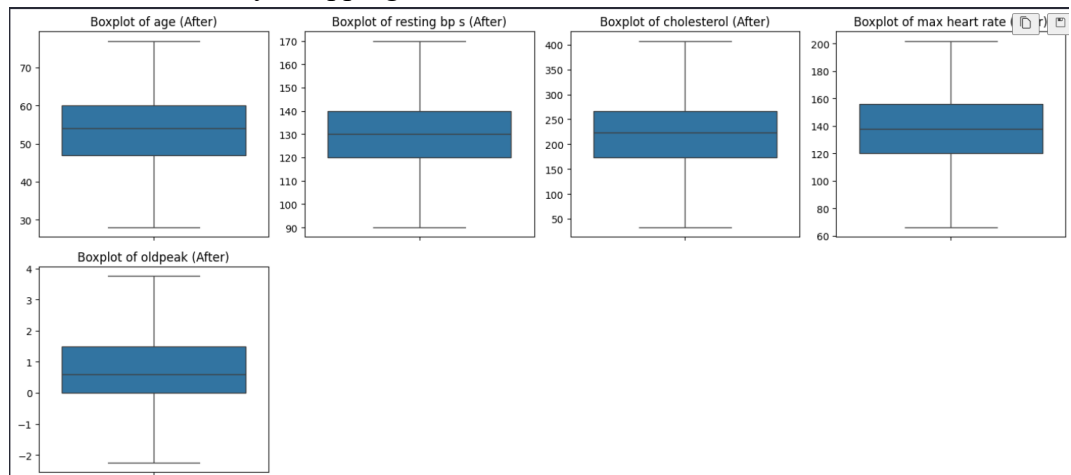
- Train-Test Split TERLEBIH DAHULU (80:20, stratified, random_state=42)



- **Penghapusan Duplikat:** 272 data duplikat dihapus untuk memastikan model mempelajari pola yang unik dan general.
- **Imputasi Median:** Nilai 0 pada kolesterol dan tekanan darah diubah menjadi NaN lalu diimputasi dengan **Median**. Median dipilih karena lebih robust terhadap outlier dibandingkan Mean.
- **Capping Outlier (IQR):** Alih-alih menghapus data ekstrem yang berharga, kami menerapkan *capping* menggunakan metode Interquartile Range (IQR). Ini menjaga distribusi data tetap dalam batas wajar tanpa kehilangan informasi.



Setelah dilakukannya Capping Outlier



- **One-Hot Encoding:** Fitur kategorikal seperti chest pain type dan ST slope diubah menjadi kolom biner. Hal ini dilakukan karena fitur tersebut tidak memiliki urutan (non-ordinal).

One-Hot Encoding dan alignment selesai diterapkan.

Preview 5 baris pertama setelah preprocessing:

	ST slope_1	ST slope_2	ST slope_3	age	chest pain type_2	chest pain type_3	chest pain type_4	cholesterol	exercise angina_1	fasting blood sugar_1	max heart rate	oldpeak	resting bp s	resting ecg_1	resting ecg_2	sex_1	target
0	0.0	1.0	0.0	63.0	1.0	0.0	0.0	217.0	1.0	1.0	128.0	1.2	139.0	1.0	0.0	1.0	1
1	1.0	0.0	0.0	55.0	1.0	0.0	0.0	214.0	0.0	1.0	180.0	0.4	110.0	1.0	0.0	1.0	0
2	0.0	1.0	0.0	59.0	0.0	0.0	1.0	338.0	1.0	1.0	130.0	1.5	130.0	1.0	0.0	0.0	1
3	0.0	1.0	0.0	47.0	0.0	0.0	1.0	238.0	1.0	0.0	124.0	0.0	160.0	0.0	0.0	1.0	1
4	0.0	1.0	0.0	50.0	0.0	0.0	1.0	238.0	1.0	1.0	139.0	0.7	145.0	0.0	0.0	1.0	1

- **StandardScaler:** Dilakukan standarisasi pada fitur numerik agar fitur dengan skala besar (seperti kolesterol) tidak mendominasi perhitungan jarak pada algoritma KNN dan SVM.

```

✓ StandardScaler diterapkan dengan BENAR (mencegah data leakage):
• Fit scaler: menggunakan HANYA X_train
• Transform: diterapkan ke X_train dan X_test
• Hasil: test set tetap independen ✓

📊 Statistik Fitur Numerik Setelah Scaling (Training Set):

```

	count	mean	std	min	25%	50%	75%	max
age	734.0	0.0	1.0007	-2.6339	-0.7260	0.1220	0.7580	2.4540
resting bp s	734.0	-0.0	1.0007	-2.2623	-0.7126	-0.1591	0.4497	3.7151
cholesterol	734.0	0.0	1.0007	-2.9523	-0.5544	-0.1301	0.4555	6.6024
max heart rate	734.0	-0.0	1.0007	-2.9587	-0.6732	0.0629	0.7601	2.5421
oldpeak	734.0	0.0	1.0007	-3.2806	-0.8191	-0.3457	0.6011	4.4828

- **SMOTE (Synthetic Minority Over-sampling Technique):** Diterapkan pada data latih untuk menyeimbangkan jumlah sampel antara pasien sehat dan sakit, sehingga model tidak bias ke kelas mayoritas.

```

✓ SMOTE berhasil diterapkan!

Distribusi kelas train setelah penanganan:
target
1      406
0      406
Name: count, dtype: int64
--- Proses SMOTE Data selesai dalam 0.78 detik ---

```

c. Pencegahan Data Leakage

Kebocoran data (*Data Leakage*) dihindari dengan protokol ketat:

- **Split First:** Pembagian data menjadi *Train* (80%) dan *Test* (20%) dilakukan di awal sebelum proses statistik apa pun.
- **Statistik Lokal:** Nilai median, batas IQR, dan parameter *StandardScaler* dihitung **hanya dari data latih**, kemudian diterapkan (*transform*) ke data uji.
- **Resampling di Data Latih:** Proses SMOTE hanya dilakukan pada data latih agar informasi dari data uji tidak bocor ke dalam proses pelatihan model.

3.5 Strategi Pembagian Data (Data Splitting)

Eksperimen ini menggunakan skema pembagian data **80% untuk pelatihan (Train)** dan **20% untuk pengujian (Test)**.

- **Stratified Sampling:** Digunakan untuk memastikan rasio kelas target (Sakit vs Sehat) tetap proporsional baik di data latih maupun data uji.
- **Justifikasi:** Mengingat jumlah data yang moderat (918 sampel), rasio 80:20 memberikan keseimbangan yang cukup antara jumlah pola yang dipelajari model dan objektivitas evaluasi akhir.

3.6 Model Baseline dan Algoritma yang Diuji

Kami membandingkan empat algoritma utama dengan satu baseline sederhana:

1. **Baseline (Dummy Classifier):** Menggunakan strategi most_frequent sebagai batas performa minimal.
2. **K-Nearest Neighbors (KNN):** Mewakili model berbasis jarak.
3. **Support Vector Machine (SVM):** Mewakili model berbasis margin optimal.
4. **Naive Bayes (Gaussian):** Mewakili model berbasis probabilitas.
5. **Decision Tree:** Mewakili model berbasis aturan (rule-based) yang intuitif.

3.7 Hyperparameter (Grid Search)

Optimasi dilakukan menggunakan GridSearchCV untuk mencari kombinasi parameter berikut:

- KNN tuned : n_neighbors=[3,5,7,9,11,13,15], weights=[uniform], metric=[euclidean,manhattan]
- NB Tuned : var_smoothing=logspace(-12,0,25)
- SVM Tuned : C=[0.1,1,10], kernel=[rbf,linear], gamma=[scale,auto]
- DT Tuned : criterion=[gini,entropy], max_depth=[3,4,5,6,None], min_samples_split=[2,5,10,20], min_samples_leaf=[1,2,4,8]

3.8 Cross Validation dan Matrik

GridSearchCV menggunakan StratifiedKFold (n_splits=10, shuffle=True, random_state=42) dengan scoring='recall'. Metrik evaluasi pada test set: Accuracy, Balanced Accuracy, **Recall (PRIORITAS)**, Precision, F1-Score (macro & weighted), AUC-ROC, Confusion Matrix.

 Ringkasan Hasil Cross-Validation (Recall):

	Recall_Mean	Recall_Std
SVM_Tuned	0.948232	0.020572
DT_Tuned	0.867317	0.058037
NB_Tuned	0.862134	0.025013
KNN_Tuned	0.726524	0.056822

Alasasn pemilihan Recall sebagai matrx utama adalah karena Risiko False Negative yang Tinggi: Dalam konteks medis, terutama untuk penyakit serius seperti penyakit jantung, kesalahan klasifikasi memiliki konsekuensi yang berbeda:

False Negative (FN): Pasien yang sebenarnya sakit diprediksi sehat. Ini adalah skenario yang paling berbahaya karena pasien tidak mendapatkan penanganan yang diperlukan, sehingga penyakitnya bisa bertambah parah, bahkan berujung fatal.

False Positive (FP): Pasien yang sebenarnya sehat diprediksi sakit. Ini bisa menyebabkan kecemasan yang tidak perlu, biaya pemeriksaan lanjutan yang mahal, atau perawatan yang tidak relevan. Meskipun tidak ideal, risiko medisnya umumnya jauh lebih rendah dibandingkan FN.

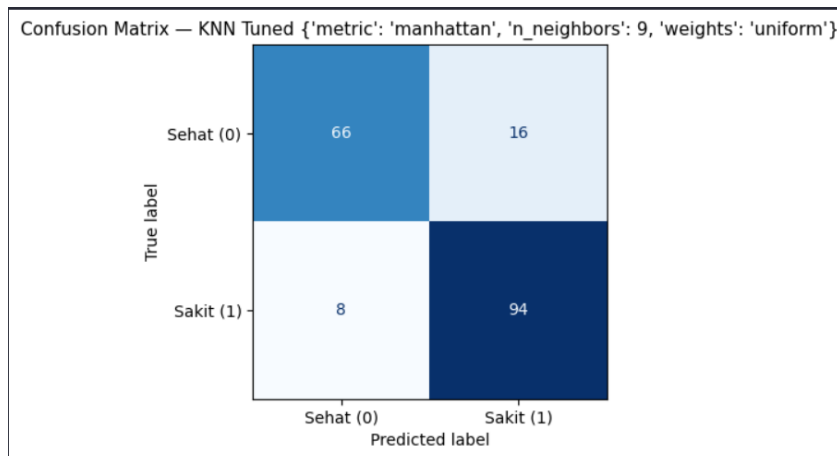
Fokus utama dalam diagnosis medis adalah meminimalkan False Negative. Kita ingin memastikan bahwa sebanyak mungkin pasien yang benar-benar sakit berhasil teridentifikasi. Recall secara langsung mengukur kemampuan model untuk mengidentifikasi semua kasus positif yang relevan (pasien sakit).

BAB 4 HASIL EKSPERIMEN

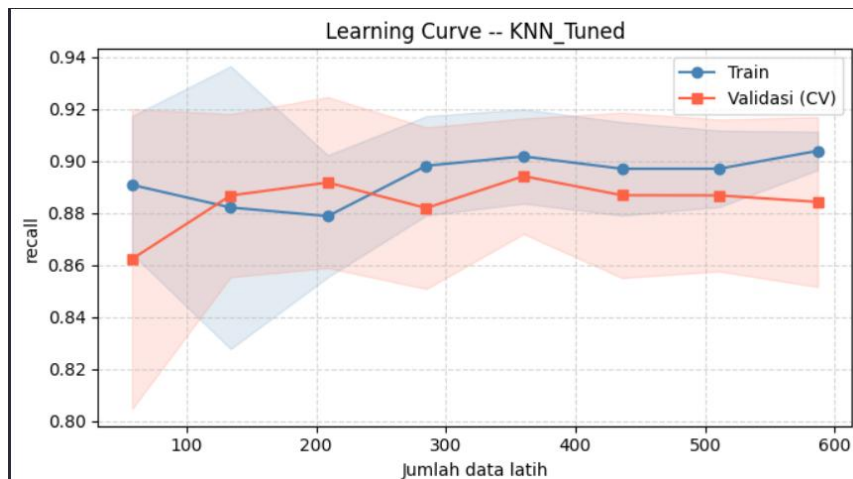
4.1 Tabel Perbandingan Semua Model

Model	Acc Test	Bal Acc	Recall	Precision	F1-Score	F1 Macro	AUC-ROC	Gap
KNN_Tuned	0.8696	0.8632	0.9216	0.8545	0.8868	0.8665	0.9231	-0.0085
KNN_Baseline	0.8641	0.8595	0.9020	0.8598	0.8804	0.8616	0.9173	0.0092
NB_Tuned	0.8587	0.8522	0.9118	0.8455	0.8774	0.8553	0.8953	-0.0181
NB_Baseline	0.8587	0.8522	0.9118	0.8455	0.8774	0.8553	0.8953	-0.0181
SVM_Tuned	0.8587	0.8451	0.9706	0.8115	0.8839	0.8517	0.9429	0.0160
SVM_Baseline	0.8587	0.8451	0.9706	0.8115	0.8839	0.8517	0.9360	0.0378
DT_Baseline	0.7065	0.6923	0.8235	0.7000	0.7568	0.6934	0.6923	0.2935
DT_Tuned	0.6467	0.6108	0.9412	0.6194	0.7471	0.5807	0.7857	0.2143

4.2 Confusion Matrix Model Terbaik (KNN Tuned)



4.3 Learning curve untuk KNN_tunning



4.4 Error Analysis

8 dari 102 pasien sakit diprediksi sehat (FN). Profil pasien yang sering terlewat memiliki chest pain type non-asymptomatic dengan oldpeak rendah dan ST slope upsloping - profil yang secara statistik mirip pasien sehat di data training. 16 pasien sehat diprediksi sakit (FP) - berdampak biaya pemeriksaan lanjutan yang tidak perlu, tapi jauh lebih aman dibanding FN dalam konteks penyakit jantung.

BAB 5

PEMBAHASAN

5.1 Pemilihan Model Terbaik

KNN Tuned ($n_neighbors=9$, $metric=manhattan$, $weights=uniform$) dipilih sebagai model terbaik berdasarkan kombinasi:

- Accuracy Test tertinggi (0.8696)
- generalisasi terbaik
- Recall tinggi (0.9216)
- hanya 8 FN dari 102 pasien sakit
- F1-Score tertinggi (0.8868)
- keseimbangan Precision-Recall terbaik
- Gap overfitting minimal (0.0086)
- model stabil

Meskipun SVM Tuned punya Recall lebih tinggi (0.9706), SVM memiliki FP jauh lebih banyak (22 FP vs 16 FP) dan Accuracy Test lebih rendah (0.8587).

5.2 Trade-off Akurasi-Kompleksitas-Interpretabilitas

Kriteria	KNN Tuned	Decision Tree Tuned	Naive Bayes Tuned	SVM Tuned
Acc Test	0.8696	0.5652 (Overfit)	0.8587	0.8587
Recall (Kelas 1)	0.9216	0.8824	0.9118	0.9706 ✓
F1 Macro	0.8665 ✓	0.4758 (Overfit)	0.8553	0.8517
FN Count	8	12	9	3 ✓
Gap (Train-Test)	-0.0085 (Baik) ✓	0.2972 (Buruk) ✗	-0.0181 (Baik) ✓	0.0160 (Baik) ✓
Kestabilan CV (Recall Std)	Sedang (± 0.08)	Rendah (± 0.11)	Sangat Stabil (± 0.02) ✓	Sangat Stabil (± 0.02) ✓
Kompleksitas	Tinggi ($O(N)$ jarak)	Rendah (If-Else)	Sangat Rendah ($P(X)$)	Tinggi ($O(N^2)$ optimasi)
Interpretabilitas	Rendah	Sangat Tinggi	Sedang	Rendah

5.3 Implikasi Institusi

Sistem ini dapat digunakan sebagai alat skrining awal di fasilitas kesehatan tingkat pertama (Puskesmas, klinik). Pasien dengan skor risiko tinggi dapat dirujuk lebih cepat ke spesialis jantung tanpa menunggu gejala memburuk. Namun, perlu diperhatikan bahwa model belum divalidasi pada populasi Indonesia secara spesifik - diperlukan fine-tuning dengan data lokal.

BAB 6

DESAIN APLIKASI

6.1 Arsitektur Backend

- src/predict.py: Pipeline inference yang memuat model (KNN Tuned), scaler, dan metadata pipeline dari folder
- models/. Preprocessing dilakukan identik dengan training: imputasi -> capping IQR -> OHE -> StandardScaler -> prediksi.
- models/best_heart_disease_model.joblib: Model KNN Tuned tersimpan (joblib)
- models/scaler.joblib: StandardScaler yang di-fit pada X_train
- models/pipeline_meta.json: Metadata (median, IQR bounds, urutan kolom)

6.2 Alur UI/UX Streamlit

- User membuka aplikasi di browser (localhost:8501)
- User mengisi 11 field input data pasien (slider, dropdown, number input)
- User menekan tombol 'Prediksi Sekarang'
- Prediksi Penyakit Jantung - Backend: preprocess input -> transform -> KNN predict -> return proba
- UI menampilkan: status (Berisiko/Tidak), probabilitas, progress bar, keterangan

6.3 Cara Menjalankan

- pip install -r requirements.txt
- streamlit run app_streamlit.py
- Buka browser di <http://localhost:8501>

BAB 7

KESIMPULAN DAN SARAN

7.1 Capaian Outcome

- Berhasil membangun pipeline ML yang bebas data leakage dengan 5 tahap preprocessing.
- KNN Tuned mencapai Accuracy 86.96%, Recall 92.16%, F1 88.68%, AUC-ROC 92.31%.
- Hyperparameter tuning dengan GridSearchCV meningkatkan recall KNN dari 90.2% ke 92.16%.
- Aplikasi Streamlit berhasil dideploy dan dapat melakukan prediksi real-time.
- Seluruh output (notebook, model, aplikasi, laporan) berhasil disel

7.2 Keterbatasan

- Dataset berasal dari populasi internasional, belum tentu representatif untuk populasi Indonesia.
- Recall 92.16% artinya masih ada ~8% pasien sakit yang terlewat (False Negative).
- Decision Tree setelah tuning mengalami penurunan akurasi signifikan (56.52%) → DT tidak cocok digunakan untuk database yang digunakan.
- Aplikasi belum terintegrasi dengan sistem rekam medis elektronik (EMR).

7.3 Rekomendasi Pengembangan

- Fine-tuning model dengan dataset penyakit jantung dari rumah sakit Indonesia.
- Eksplorasi model ensemble (Random Forest, XGBoost) untuk meningkatkan Recall.
- Implementasi SHAP values untuk explainability, penting untuk kepercayaan klinisi.
- Integrasi dengan sistem EMR untuk workflow yang lebih seamless.
- Penambahan fitur troponin dan BNP yang lebih prediktif untuk diagnosis jantung.

Lampiran

a. Kode Penting

- Melakukan Pengecekan Apakah terdapat outlier atau tidak

```
import matplotlib.pyplot as plt
import seaborn as sns

# Kolom numerik murni yang boleh dicek outlier
numeric_cols = ['age', 'resting bp s', 'cholesterol',
                'max heart rate', 'oldpeak']

# Kolom kategoris yang TIDAK boleh dicek outlier
# (meskipun tipenya int64, nilainya adalah kode kategori)
categorical_cols = ['sex', 'chest pain type', 'fasting blood sugar',
                   'resting ecg', 'exercise angina', 'ST slope']

print(f"Kolom yang akan dicek outlier: {numeric_cols}")

plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(3, 4, i) # Adjust subplot grid as needed
    sns.boxplot(y=df[col])
    plt.title(f'Boxplot of {col}')
    plt.ylabel('')
plt.tight_layout()
plt.show()
```

- Langkah Preprocessing

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

#Penghapusan Duplikat Data
df = df.drop_duplicates()
print(f"✅ Data duplikat berhasil dihapus!")
print(f"🔥 Jumlah data duplikat: {df.duplicated().sum()}")

# Split dilakukan TERLEBIH DAHULU agar data test tetap independen
df_raw = df.copy()
X = df_raw.drop(columns=['target'])
y = df_raw['target']

X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Preprocessing dilakukan secara terpisah pada train dan test
numeric_cols = ['age', 'resting bp s', 'cholesterol', 'max heart rate', 'oldpeak']
categorical_cols = ['sex', 'chest pain type', 'fasting blood sugar', 'resting ecg', 'exercise angina', 'ST slope']

def preprocess_train_test(X_train_raw, X_test_raw):
    X_train = X_train_raw.copy()
    X_test = X_test_raw.copy()

    # 1) Imputasi: nilai 0 yang tidak masuk akal diubah menjadi NaN
    for col in ['cholesterol', 'resting bp s']:
        X_train[col] = X_train[col].replace(0, np.nan)
        X_test[col] = X_test[col].replace(0, np.nan)

    # 2) Imputasi dengan median dari data TRAIN saja
    train_medians = X_train[numeric_cols].median()
    X_train[numeric_cols] = X_train[numeric_cols].fillna(train_medians)
    X_test[numeric_cols] = X_test[numeric_cols].fillna(train_medians)
```



```

# 3) Outlier capping dengan batas IQR dari data TRAIN saja
# Fungsi untuk menangani outlier menggunakan metode capping (IQR)
def cap_outliers_iqr(df_col):
    Q1 = df_col.quantile(0.25)
    Q3 = df_col.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    return df_col.clip(lower=lower_bound, upper=upper_bound)

# Aplikasikan capping pada setiap kolom numerik
print("Menerapkan capping outlier pada kolom numerik...")
for col in numeric_cols:
    # Skip 'target' column if it's in numeric_cols, as it's a categorical label
    if col == 'target':
        continue
    df[col] = cap_outliers_iqr(df[col])
    print(f"    ✅ Outlier pada kolom '{col}' telah ditangani.")

print("\nOutlier berhasil ditangani untuk semua fitur numerik (kecuali 'target').")

# 4) One-Hot Encoding dengan alignment kolom antar train dan test
X_train_enc = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True, dtype=int)
X_test_enc = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True, dtype=int)
X_train_enc, X_test_enc = X_train_enc.align(X_test_enc, join='outer', axis=1, fill_value=0)
return X_train_enc.astype(float), X_test_enc.astype(float)

X_train, X_test = preprocess_train_test(X_train_raw, X_test_raw)
df_encoded = pd.concat([X_train.assign(target=y_train), X_test.assign(target=y_test)], ignore_index=True)

print('✅ Split dilakukan sebelum preprocessing')
print('✅ Imputasi, capping, dan encoding menggunakan statistik dari data train saja')
print(f'    Train shape: {X_train.shape}')
print(f'    Test shape : {X_test.shape}')

```

- Kode untuk melatih model baseline

```

# === Soal 03: Baseline KNN, Naive Bayes, SVM (+ optional Decision Tree) ===
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier

# Definisikan model baseline (default params) - Decision Tree tetap disimpan seperti permintaan
models_baseline = {
    'KNN' : KNeighborsClassifier(),
    'GaussianNB' : GaussianNB(),
    'SVM' : SVC(probability=True, random_state=42),
    'DecisionTree' : DecisionTreeClassifier(random_state=42),
}

# Jalankan evaluasi untuk setiap model baseline dan simpan hasilnya
print('\n--- Menjalankan baseline models: KNN, GaussianNB, SVM, DecisionTree ---')
start_time = time.time()
for name, mdl in models_baseline.items():
    try:
        hasil_semua[name] = evaluasi_model(name, mdl, X_train, X_test, y_train, y_test)
    except Exception as e:
        print(f'⚠️ Error saat melatih {name}:', e)

end_time = time.time()
print(f'--- Semua baseline models selesai dalam {end_time - start_time:.2f} detik ---')

# Tampilkan ringkasan hasil singkat
import pandas as _pd
df_hasil = _pd.DataFrame.from_dict([k:v for k,v in hasil_semua.items() if isinstance(v, dict)], orient='index')
display(df_hasil[['Acc Train', 'Acc Test', 'Bal Acc', 'Recall', 'Precision', 'F1-Score', 'AUC-ROC']].sort_values('Acc Test', ascending=False))

```

- Kode untuk Hyperparameter tuning best model

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
import time

print('\n--- Memulai Hyperparameter Tuning KNN ---')
start_time_knn = time.time()

param_grid_knn = {
    'n_neighbors': [3,6,7,9],
    'weights': ['uniform'],
    'metric': ['euclidean', 'manhattan'],
}

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

# Gunakan X_train_res jika tersedia (SMOTE), fallback ke X_train
try:
    X_fit_knn, y_fit_knn = X_train_res, y_train_res
    print('INFO: menggunakan X_train_res (SMOTE)')
except NameError:
    X_fit_knn, y_fit_knn = X_train, y_train
    print('INFO: menggunakan X_train (tanpa SMOTE)')

gs_knn = GridSearchCV(
    KNeighborsClassifier(),
    param_grid_knn,
    cv=cv,
    scoring='recall',
    n_jobs=-1,
    verbose=0,
)

gs_knn.fit(X_fit_knn, y_fit_knn)

end_time_knn = time.time()
print(f'Parameter terbaik KNN : {gs_knn.best_params_}')
print(f'Recall CV terbaik : {gs_knn.best_score_:.4f}')
print(f'--- Hypertuning KNN selesai dalam {end_time_knn - start_time_knn:.2f} detik ---')

# Evaluasi pada test set
knn_tuned = gs_knn.best_estimator_
hasil_semua['KNN Tuned'] = evaluasi_model(
    f'KNN Tuned {gs_knn.best_params_}',
    knn_tuned, X_train, X_test, y_train, y_test
)

# Tampilkan ringkasan semua model termasuk Bal Acc
import pandas as pd
df_hasil = pd.DataFrame.from_dict(
    [k: v for k, v in hasil_semua.items() if isinstance(v, dict)], orient='index'
)
display(df_hasil[['Acc Train', 'Acc Test', 'Bal Acc', 'Recall', 'Precision', 'F1-Score', 'AUC-ROC']])
df_hasil.sort_values('Acc Test', ascending=False)
```

b. Resource Teknis

Link Repositori : <https://github.com/al-Fajrii/KlasifikasiHeartDisease.git>

Reuquirements : numpy, pandas, scikit-learn, imbalanced-learn, joblib, scipy, matplotlib, seaborn, streamlit, gradio, jupyter, ipykernel

Bukti demo dengan Link Presentasi : <https://youtu.be/y-PM2wRXrmc>

c. Antar Muka Aplikasi Streamlit untuk klasifikasi heart disease

Deploy



Prediksi Risiko Penyakit Jantung

Aplikasi ini menggunakan model **K-Nearest Neighbors (KNN Tuned)** yang dilatih pada dataset Heart Disease untuk memprediksi risiko penyakit jantung pasien.

⚠ Disclaimer: Prediksi ini bersifat pendukung keputusan, bukan diagnosis medis. Selalu konsultasikan hasil dengan dokter / tenaga medis profesional.



Data Pasien

Usia (tahun)

55

-

+

Hasil EKG istirahat

0 - Normal

▼

Jenis Kelamin

Laki-laki

▼

Detak Jantung Maksimum (bpm)

130

-

+

Tipe Nyeri Dada

4 - Asymptomatic

▼

Angina Saat Olahraga

Tidak Ada

▼

Tekanan Darah Sistolik Istirahat (mmHg)

140

- +

Oldpeak — Depresi Segmen ST (mm)

2,0

- +

Kolesterol Total (mg/dL)

250

- +

Slope Segmen ST

2 — Flat

▼

Gula Darah Puasa

≤ 120 mg/dL (Normal)

▼

Prediksi Sekarang

Hasil Prediksi

TIDAK BERISIKO PENYAKIT JANTUNG

Probabilitas Sehat

66.7%

Probabilitas Sakit

33.3%

Risiko penyakit jantung

✓

Pasien diprediksi TIDAK berisiko penyakit jantung. Keyakinan model: 66.7%. → Tetap lakukan pemantauan rutin tekanan darah & kolesterol.

Model: KNN Tuned (n_neighbors=9, metric=manhattan, weights=uniform) | Dataset: Heart Disease — Mendeley (CC BY 4.0)

© 2025 Muhammad Syaifq Al Fajri — UAS Machine Learning A11.2024.15698 | Universitas Dian Nuswantoro

DAFTAR PUSTAKA

Mohammed Amine Bouqentar, O. T. (2024). Early heart disease prediction using feature engineering and machine learning algorithms. *Heliyon*, 23.